

Polycopié de cours : Langage et programmation (Shell, Python, Java, JEE)

Public concerné :

Étudiants de Licence 3 en Génie Logiciel ou Architecture des Systèmes et Réseaux (ASR) au Cameroun.

Volume horaire : 4 séances × 4 heures = 16 heures (environ 10 heures de travaux pratiques).

Objectifs généraux

À l'issue de ces 16 heures, l'étudiant sera capable de :

- Comprendre le rôle et l'utilisation de quatre langages complémentaires dans le développement logiciel.
- Écrire des scripts shell pour automatiser des tâches d'administration système.
- Développer de petits programmes en Python pour le traitement de données et l'interaction avec le système.
- Créer des applications console simples en Java en utilisant les principes de la programmation orientée objet.
- Construire une mini-application web avec les technologies JEE (Servlets et JSP).
- Relier ces compétences dans un mini-projet intégrateur.

Organisation du cours

Chaque séance alterne un bref rappel théorique (45 minutes) et un long atelier de codage (3h15). Tous les extraits de code sont expliqués ligne par ligne. Les exercices sont corrigés et commentés.

Environnement technique nécessaire

- Pour le Shell : terminal Linux (Ubuntu recommandé) ou bien Windows Subsystem for Linux (WSL) sous Windows. Nous utilisons le shell Bash (Bourne Again SHell).
- Pour Python : Python 3.8 ou supérieur, un éditeur de code (VS Code, PyCharm) ou un simple bloc-notes.
- Pour Java/JEE : JDK 11 ou supérieur (le JDK contient le compilateur javac et la machine virtuelle java), un environnement de développement intégré comme Eclipse ou IntelliJ IDEA, un serveur d'applications web léger comme Apache Tomcat (version 9 ou 10), et une base de données (H2, MySQL ou SQLite). Pour simplifier, nous utiliserons H2 qui s'exécute en mémoire.

Introduction: Pourquoi apprendre ces quatre langages ?

Le paysage des langages de programmation

Dans le domaine du génie logiciel et de l'architecture des systèmes, aucun langage unique ne peut répondre à tous les besoins. Chaque langage possède des forces et des contextes d'utilisation privilégiés. Ce cours vous propose de découvrir quatre langages majeurs selon une approche pragmatique : Shell, Python, Java et JEE (Java Enterprise Edition, qui est en réalité une plateforme plutôt qu'un langage, mais qui utilise Java comme langage de base).

1. Pourquoi le Shell (Bash) ?

Le shell est l'interface entre l'utilisateur et le noyau d'un système Unix/Linux. Apprendre à écrire des scripts shell permet d'automatiser des tâches répétitives : sauvegarde de fichiers, surveillance des ressources, traitement de logs, envoi de mails d'alerte, etc. Dans une carrière de développeur ou d'administrateur système, la maîtrise du shell est indispensable pour être efficace en ligne de commande.

2. Pourquoi Python ?

Python est un langage de haut niveau, très lisible, qui excelle dans le prototypage rapide, l'analyse de données, l'intelligence artificielle, l'automatisation et le scripting avancé. Contrairement au shell, Python dispose de structures de données évoluées (listes, dictionnaires) et d'une gestion d'erreur robuste. Il permet d'écrire des programmes plus complexes sans souffrir des limitations du shell (comme la lenteur sur les calculs ou la syntaxe parfois ésotérique).

3. Pourquoi Java ?

Java est un langage compilé (vers un bytecode exécuté par une machine virtuelle) orienté objet, extrêmement utilisé dans les applications d'entreprise, la programmation mobile (Android) et les serveurs. Sa rigueur (typage statique, gestion explicite des exceptions, forte encapsulation) en fait un excellent langage pour apprendre les concepts solides du génie logiciel. De plus, Java est le langage natif de la plateforme JEE.

4. Pourquoi JEE (Servlets/JSP) ?

JEE n'est pas un langage, mais un ensemble de spécifications qui étendent Java pour le développement web et les applications d'entreprise. Les Servlets sont des objets Java qui répondent à des requêtes HTTP. Les JSP (JavaServer Pages) permettent de mélanger du HTML avec du code Java pour générer dynamiquement des pages web. Comprendre ces bases est le premier pas vers la création de sites web dynamiques et d'APIs REST.

Chapitre 1 : Programmation Shell (Bash)

Objectifs détaillés

- Comprendre ce qu'est un shell, un terminal, un script.
- Connaître la structure d'un script Bash (shebang, commentaires, droits d'exécution).
- Utiliser les variables, les opérateurs de test, les structures conditionnelles (if-elif-else) et les boucles (for, while).
- Écrire des fonctions et manipuler les paramètres.
- Appliquer ces concepts dans des exercices pratiques : surveillance de disque, analyse de logs.

1.1 Qu'est-ce que le shell ?

Le shell est un interpréteur de commandes. Quand vous ouvrez un terminal, le shell affiche un prompt (par exemple \$) et attend que vous tapiez des commandes (ls, cd, mkdir...). Le shell le plus répandu sous Linux est Bash (Bourne Again SHell). Un script shell est un fichier texte contenant une série de commandes, comme une recette de cuisine. En rendant ce fichier exécutable, le shell lira et exécutera les commandes les unes après les autres.

1.2 Votre premier script – explication ligne par ligne

Créez un fichier premier.sh avec le contenu suivant :

```
``bash
#!/bin/bash
# Ceci est mon premier script
echo "Bonjour, monde !"
``
```

Explication détaillée :

- **#!/bin/bash** : c'est le shebang (ou "sha-bang"). Il indique au système quel interpréteur utiliser pour exécuter ce script. Ici, on demande le programme /bin/bash. Sans cette ligne, le shell par défaut (souvent Bash) essaiera d'interpréter le script, mais il est recommandé de toujours la mettre.

· **#** Ceci est mon premier script : tout ce qui suit un **#** (sauf le shebang) est un commentaire. Le shell ignore les commentaires ; ils servent à documenter le code.

· **echo "Bonjour, monde !"** : la commande echo affiche son argument à l'écran. Les guillemets doubles permettent d'afficher des espaces et des variables à l'intérieur. Ici, le message "Bonjour, monde !" s'affichera dans le terminal.

Exécution :

```
```bash
```

```
$ chmod +x premier.sh # ajoute le droit d'exécution (x) au fichier
```

```
$./premier.sh # exécute le script depuis le répertoire courant
```

```
```
```

chmod change les permissions ; +x signifie "ajouter exécutable". ./ indique que le fichier se trouve dans le répertoire courant (pour des raisons de sécurité, le shell ne cherche pas les exécutables dans le répertoire courant par défaut).

1.3 Les variables

Une variable est un espace mémoire nommé qui peut stocker une valeur (chaîne de caractères, nombre, etc.). En Bash, les variables n'ont pas de type explicite.

```
```bash
```

```
#!/bin/bash
```

```
nom="Alice" # assignation : pas d'espace autour de '='
```

```
age=25
```

```
echo "Je m'appelle $nom et j'ai $age ans"
```

```
```
```

Explications :

· **nom="Alice"** : le nom de la variable est nom. On lui affecte la chaîne "Alice". Il est interdit de mettre des espaces autour du = (contrairement à Java ou Python).

· **echo "Je m'appelle \$nom ..."** : le caractère \$ devant le nom de la variable demande au shell de la substituer par sa valeur. Ainsi, l'argument de echo devient Je m'appelle Alice et j'ai 25 ans.

· Pour concaténer sans espace : **\${nom}_suffixe**. Les accolades délimitent le nom de la variable si nécessaire.

Variables spéciales :

- \$0, \$1, \$2... : paramètres du script (le nom du script lui-même, premier argument, second argument...)
- \$# : nombre d'arguments passés
- \$? : code de retour de la dernière commande exécutée (0 = succès, non nul = erreur)

1.4 Structures de contrôle – if expliqué en détail

```
```bash
#!/bin/bash

read -p "Entrez votre âge : " age

if [$age -ge 18]; then
 echo "Vous êtes majeur."
elif [$age -ge 0]; then
 echo "Vous êtes mineur."
else
 echo "Âge invalide."
fi
```
```

Explication ligne par ligne :

- read -p "Entrez votre âge : " age : la commande read attend une saisie clavier. L'option -p permet d'afficher un message sans avoir à faire un echo séparé. Le texte saisi est stocké dans la variable age.
- if [\$age -ge 18]; then : if introduit une condition. Les crochets [] sont une commande (alias de test) qui évalue une expression. Attention : il faut des espaces après [et avant]. -ge signifie "greater or equal" (>=). Si la condition est vraie, le shell exécute les commandes jusqu'au then (le then peut être sur la ligne suivante, mais on le met souvent après ;).
- echo "Vous êtes majeur." : exécuté si la condition est vraie.
- elif [\$age -ge 0]; then : "else if". Si la première condition est fausse, on teste celle-ci.
- else : si aucune condition précédente n'est vraie.
- fi : termine le bloc if (c'est "if" inversé).

Tableau des principaux opérateurs de test :

| Opérateur | Signification | Exemple |
|-----------|---|----------------------|
| -eq | Égal (numérique) | ["\$a" -eq "\$b"] |
| -ne | Différent (numérique) | ["\$a" -ne "\$b"] |
| -lt | Inférieur (<) | ["\$a" -lt "\$b"] |
| -le | Inférieur ou égal (<=) | ["\$a" -le "\$b"] |
| -gt | Supérieur (>) | ["\$a" -gt "\$b"] |
| -ge | Supérieur ou égal (>=) | ["\$a" -ge "\$b"] |
| = | Égal (chaînes) | ["\$a" = "\$b"] |
| != | Différent (chaînes) | ["\$a" != "\$b"] |
| -f | Vrai si le fichier existe et est régulier | [-f "/etc/passwd"] |
| -d | Vrai si le répertoire existe | [-d "/home"] |

Remarque : Les guillemets autour des variables sont importants pour que le test fonctionne correctement si la variable est vide ou contient des espaces.

1.5 La boucle for – explications

```
```bash
```

```
#!/bin/bash
```

```
for fichier in *.txt; do
```

```
 echo "Traitement de $fichier"
```

```
 wc -l "$fichier"
```

```
done
```

```
```
```

Détail :

- for fichier in *.txt : le motif *.txt est expansé par le shell en une liste de tous les fichiers se terminant par .txt dans le répertoire courant. La boucle affecte chaque nom de fichier à la variable fichier, l'un après l'autre.

- do : marque le début du bloc à répéter.

- echo "Traitement de \$fichier" : affiche le nom du fichier en cours.
- wc -l "\$fichier" : la commande wc -l compte le nombre de lignes du fichier. On met "\$fichier" entre guillemets pour éviter les problèmes si le nom contient des espaces.
- done : fin de la boucle.

1.6 La boucle while – explications

```
```bash
#!/bin/bash

compteur=1

while [$compteur -le 5]; do
 echo "Itération $compteur"

 ((compteur++))
done
...
```
```

((compteur++)) est une arithmétique shell qui incrémente compteur de 1. Contrairement à \$(()) qui évalue une expression, la double parenthèse avec let permet de modifier la variable. ((compteur++)) équivaut à compteur=\$((compteur+1)).

1.7 Les fonctions

```
```bash
#!/bin/bash

afficher_date() {
 echo "Nous sommes le $(date +%d/%m/%Y)"
}

afficher_date
...
```
```

Fonctionnement :

- afficher_date() { ... } : définit une fonction nommée afficher_date. Les parenthèses sont vides car elle ne prend pas de paramètres formels (en Bash, on utilise \$1, \$2 à l'intérieur de la fonction pour accéder aux arguments).

- `$(date +%d/%m/%Y)` : substitution de commande. La commande entre `$( )` est exécutée, et sa sortie (stdout) remplace l'expression. Ici, `date +%d/%m/%Y` produit une chaîne comme "18/03/2025".

- `afficher_date` : appelle la fonction.

1.8 Expressions régulières et grep

grep est un outil puissant pour chercher du texte avec des motifs. Exemple :

```
```bash
```

```
$ grep "^[0-9]" fichier.txt # lignes commençant par un chiffre
```

```
$ grep "error|fail" log.txt # lignes contenant "error" ou "fail"
```

```
...
```

- `^` : début de ligne.

- `[0-9]` : un seul chiffre.

- `|` : ou (alternation) – il faut l'échapper avec `\` dans les grep basiques. Sous grep -E, on peut écrire `error|fail`.

## 1.9 Exercices pratiques avec corrigés détaillés

### Exercice 1 : info\_systeme.sh

Énoncé : Écrire un script qui affiche le nom d'utilisateur, le nom de machine, la date/heure et l'espace disque disponible sur la partition racine.

#### Corrigé commenté :

```
```bash
```

```
#!/bin/bash
```

```
# Affiche le nom de l'utilisateur courant
```

```
echo "Utilisateur : $(whoami)"
```

```
# $(whoami) exécute la commande whoami et insère sa sortie
```

```
echo "Machine : $(hostname)"
```



```
# hostname donne le nom de la machine
echo "Date/Heure : $(date)"
# date sans argument donne le jour, mois, année, heure, fuseau
echo "Espace disque :"
df -h / | tail -1
# df -h : espace disque en format lisible (Ko, Mo, Go)
# L'argument "/" limite à la partition racine
# tail -1 : prend la dernière ligne (la première ligne est l'en-tête)
...
```

Exercice 2 : surveillance.sh (alerte disque)

Énoncé : Si l'utilisation du disque dépasse 80%, envoyer une alerte dans les logs système.

Corrigé commenté :

```
``bash
#!/bin/bash
# Récupère le pourcentage d'utilisation de la partition racine
# df -h / | grep "/" : on extrait la ligne qui contient le chemin "/"
# awk '{print $5}' : imprime le 5ème champ (la colonne du pourcentage)
# sed 's/%//' : supprime le caractère '%'
utilisation=$(df / | grep "/" | awk '{print $5}' | sed 's/%//')
# Vérifie si la valeur est supérieure à 80
if [ $utilisation -gt 80 ]; then
    # logger envoie le message dans le journal système (/var/log/syslog)
    echo "Alerte : Espace disque à ${utilisation}%" | logger -t surveillance
fi
...
```

Alternative : afficher un message sur le terminal de tous les utilisateurs avec wall ou envoyer un mail.

Chapitre 2 : Programmation Python

Objectifs détaillés

- Connaître la syntaxe de Python (pas d'accolades, indentation significative).
- Manipuler les types natifs : entiers, flottants, chaînes, listes, dictionnaires.
- Maîtriser les structures de contrôle (if, for, while).
- Écrire des fonctions et gérer les exceptions (try/except).
- Utiliser la programmation orientée objet : classes, héritage, encapsulation.
- Lire/écrire des fichiers et appeler des commandes système.

2.1 Premiers pas – différence avec Shell

Python est interprété, mais contrairement au shell, il possède une syntaxe riche pour les structures de données et la logique.

```
```python
hello.py
print("Bonjour, monde !")
...`
```

Exécution : `python3 hello.py`. Python est sensible à la casse et à l'indentation (les blocs sont délimités par des espaces ou des tabulations, pas par des accolades).

### 2.2 Variables et types – explications détaillées

```
```python
# Nombres
age = 25                # int (entier)
taille = 1.75           # float (virgule flottante)

# Chaînes de caractères
nom = "Alice"
message = f"Je m'appelle {nom} et j'ai {age} ans"
...`
```

Les f-strings (chaînes formatées) commencent par f. Les {} sont remplacés par les valeurs des variables.

Listes (équivalent des tableaux dynamiques)

```
fruits = ["pomme", "banane", "orange"]
```

```
fruits.append("kiwi")    # ajoute un élément à la fin
```

```
print(fruits[0])        # accès par indice : affiche "pomme"
```

Dictionnaires (équivalent des tables de hachage / objets JSON)

```
personne = {"nom": "Dupont", "age": 30}
```

```
print(personne["nom"])   # accès par clé
```

```
...
```

Explications supplémentaires :

- Python est dynamiquement typé : on n'a pas besoin de déclarer le type d'une variable.
- Les chaînes peuvent être entourées de guillemets simples ' ou doubles ". Les f-strings nécessitent le préfixe f et les guillemets (simples ou doubles).
- Les listes sont mutables : on peut modifier, ajouter, supprimer des éléments.
- Les dictionnaires associent des clés (immutables) à des valeurs quelconques.

2.3 Structures de contrôle – explication de l'indentation

```
```python
```

```
age = int(input("Entrez votre âge: ")) # input() lit une chaîne, int() la convertit en nombre
```

```
if age >= 18:
```

```
 print("Majeur")
```

```
elif age >= 0:
```

```
 print("Mineur")
```

```
else:
```

```
 print("Âge invalide")
```

```
...
```

### **Points clés :**

- L'indentation (généralement 4 espaces) indique le bloc d'instructions.

- Les deux-points : terminent l'en-tête de la structure (if, elif, else, for, etc.).
- elif signifie "else if".
- input() retourne toujours une chaîne ; on convertit avec int().

### **Boucle for :**

```
```python
for i in range(5):
    print(f"Itération {i}")
```
```

range(5) génère une séquence de 0 à 4 (exclu). C'est un itérateur.

```
```python
for fruit in fruits:
    print(fruit)
```
```

La boucle for itère directement sur les éléments de la liste fruits.

### **Boucle while :**

```
```python
compteur = 0
while compteur < 5:
    print(compteur)
    compteur += 1 # incrémentation
```
```

Attention à ne pas oublier l'incrément, sinon la boucle est infinie.

## **2.4 Fonctions et gestion des exceptions**

```
```python
def diviser(a, b):
    """Divise a par b et gère la division par zéro."""
    try:
```

```

    return a / b
except ZeroDivisionError:
    print("Erreur : Division par zéro")
    return None
resultat = diviser(10, 2)
print(resultat) # 5.0
...

```

Explications :

- def définit une fonction. Les paramètres a et b sont passés par valeur (ou plutôt par référence d'objet mais c'est un détail avancé).
- La chaîne entre triples guillemets est une docstring (documentation).
- try: : on essaie d'exécuter le code.
- except ZeroDivisionError: : on attrape l'exception particulière si elle se produit.
- return renvoie une valeur ; si on renvoie None, c'est équivalent à ne rien renvoyer (implicitement None).

2.5 Programmation orientée objet (POO) détaillée

```

```python
class Personne:
 def __init__(self, nom, age):
 self.nom = nom # attribut public
 self.__age = age # attribut privé (convention : double underscore)
 def sePresenter(self):
 return f"Je m'appelle {self.nom}, j'ai {self.__age} ans"
 def get_age(self):
 return self.__age

class Etudiant(Personne):
 def __init__(self, nom, age, matricule):
 super().__init__(nom, age) # appel au constructeur de la classe parente

```

```

 self.matricule = matricule

 def sePresenter(self): # redéfinition (override)
 return f"{super().sePresenter()}, matricule {self.matricule}"

p = Personne("Alice", 25)

print(p.sePresenter()) # affiche : Je m'appelle Alice, j'ai 25 ans
...

```

### Explications détaillées :

- `class Personne`: définit une classe. Par convention, les noms de classes commencent par une majuscule.
- `__init__` est le constructeur. Il est appelé automatiquement lors de la création d'un objet. `self` représente l'instance courante (équivalent de `this` en Java).
- `self.nom = nom` : crée un attribut d'instance. Par défaut, les attributs sont publics (accessibles depuis l'extérieur).
- `self.__age = age` : avec deux underscores devant, Python mangle le nom (le rend privé) – c'est une encapsulation forte. On y accède depuis l'extérieur via `_Personne__age`, mais c'est déconseillé.
- La méthode `get_age()` permet un accès contrôlé.
- `class Etudiant(Personne)` : héritage. `Etudiant` hérite de `Personne`.
- `super().__init__(nom, age)` : appelle le constructeur de la classe parente pour initialiser la partie `Personne`.
- `sePresenter` redéfinit la méthode héritée. On utilise `super().sePresenter()` pour appeler la version parente.

## 2.6 Manipulation de fichiers – explications

```

```python
# Écriture

with open("data.txt", "w") as fichier:
    fichier.write("Ligne 1\n")
    fichier.write("Ligne 2\n")
...

```

- `open("data.txt", "w")` : ouvre le fichier en écriture ("w"). Si le fichier existe, il est écrasé. Retourne un objet fichier.
- `with ... as fichier :` gestionnaire de contexte. Il garantit que le fichier est correctement fermé même si une exception survient.
- `fichier.write()` écrit la chaîne dans le fichier. Les retours à la ligne doivent être ajoutés explicitement (`\n`).

```
```python
```

```
Lecture
```

```
with open("data.txt", "r") as fichier:
```

```
 for ligne in fichier:
```

```
 print(ligne.strip()) # strip() enlève les espaces et les retours à la ligne
```

```
```
```

- "r" : lecture. On peut itérer directement sur l'objet fichier pour obtenir chaque ligne.

```
```python
```

```
import json
```

```
data = {"nom": "Dupont", "scores": [10, 20, 15]}
```

```
with open("data.json", "w") as f:
```

```
 json.dump(data, f, indent=4)
```

```
```
```

- `json.dump()` sérialise l'objet Python en JSON et l'écrit dans le fichier. `indent=4` pour une sortie formatée.

2.7 Exercices pratiques avec corrigés commentés

Exercice 1 : statistiques.py

Énoncé : Lire un fichier `nombres.txt` contenant un entier par ligne, afficher somme, moyenne, min, max.

Corrigé avec explications :

```
```python
def analyser_fichier(nom_fichier):
 """Fonction qui lit un fichier d'entiers et affiche des statistiques."""
 try:
 # "r" : lecture
 with open(nom_fichier, 'r') as f:
 # On lit toutes les lignes, on enlève les espaces, on ne garde que celles non vides
 # On convertit chaque ligne en int
 nombres = [int(ligne.strip()) for ligne in f if ligne.strip()]
 if nombres:
 print(f"Somme: {sum(nombres)}")
 print(f"Moyenne: {sum(nombres)/len(nombres):.2f}") # .2f : deux décimales
 print(f"Min: {min(nombres)}")
 print(f"Max: {max(nombres)}")
 else:
 print("Fichier vide")
 except FileNotFoundError:
 print(f"Fichier {nom_fichier} introuvable")
```
```

- La compréhension de liste `[int(ligne.strip()) for ligne in f if ligne.strip()]` est plus concise qu'une boucle.
- `sum`, `min`, `max` sont des fonctions intégrées.
- `.2f` dans la f-string formate le nombre avec deux chiffres après la virgule.

Exercice 2 : classe CompteBancaire

Énoncé : Créer une classe avec dépôt, retrait, affichage de solde.

Corrigé commenté :

```
```python
class CompteBancaire:
 def __init__(self, solde_initial=0):
 """Constructeur : solde initial par défaut 0."""
 self.solde = solde_initial # attribut public
 def deposer(self, montant):
 """Ajoute un montant au solde si celui-ci est positif."""
 if montant > 0:
 self.solde += montant
 print(f"Dépôt de {montant}€ effectué")
 else:
 print("Montant invalide (doit être positif)")
 def retirer(self, montant):
 """Retire un montant si le solde est suffisant."""
 if 0 < montant <= self.solde:
 self.solde -= montant
 print(f"Retrait de {montant}€ effectué")
 else:
 print("Solde insuffisant ou montant invalide")

 def afficher_solde(self):
 print(f"Solde actuel: {self.solde}€")
...
```
```

- solde_initial=0 : paramètre optionnel. Si on crée un compte sans argument, le solde sera 0.
- La validation des montants évite les retraits négatifs ou les dépôts négatifs.

Chapitre 3 : Programmation Java

Objectifs détaillés

- Comprendre la compilation Java (.java → .class → machine virtuelle).
- Maîtriser la syntaxe : classes, main, types primitifs, objets.
- Utiliser les structures de contrôle, les tableaux, les collections (ArrayList, HashMap).
- Appliquer la POO : encapsulation, héritage, polymorphisme.
- Gérer les exceptions avec try-catch-finally.
- Se connecter à une base de données avec JDBC.

3.1 Structure d'un programme Java – explications ligne par ligne

```
``java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Bonjour, monde !");
    }
}
``
```

- `public class HelloWorld` : déclare une classe publique nommée HelloWorld. Le nom du fichier doit être HelloWorld.java (même casse).
- `public static void main(String[] args)` : le point d'entrée du programme. `public` : accessible depuis l'extérieur ; `static` : la méthode appartient à la classe et non à une instance ; `void` : ne renvoie rien ; `String[] args` : tableau des arguments de la ligne de commande.
- `System.out.println()` : affiche une ligne sur la sortie standard. `System` est une classe, `out` est un attribut statique de type `PrintStream`, `println` est une méthode.

Compilation et exécution :

```
```bash
$ javac HelloWorld.java # produit HelloWorld.class
$ java HelloWorld # lance la JVM sur la classe HelloWorld
...`
```

## 3.2 Variables et types primitifs – explications

```
```java
public class Variables {
    public static void main(String[] args) {
        int age = 25;           // entier signé 32 bits
        double taille = 1.75;   // nombre à virgule flottante 64 bits
        boolean estMajeur = true; // vrai/faux
        char lettre = 'A';      // caractère Unicode 16 bits
        String nom = "Alice";   // chaîne de caractères (objet, pas primitif)
        System.out.println(nom + " a " + age + " ans.");
    }
}
...`
```

- L'opérateur + concatène les chaînes et convertit automatiquement les primitifs en chaînes.
- Le String est immuable.

3.3 Structures de contrôle – syntaxe proche de C/C++

```
```java
public class Controle {
 public static void main(String[] args) {
 int age = 18;
 if (age >= 18) {
 ...
 }
 }
}
...`
```

```

 System.out.println("Majeur");
 } else {
 System.out.println("Mineur");
 }
 for (int i = 0; i < 5; i++) {
 System.out.println("i = " + i);
 }
 int compteur = 0;
 while (compteur < 5) {
 System.out.println(compteur);
 compteur++;
 }
 int[] nombres = {10, 20, 30, 40, 50};
 for (int n : nombres) {
 System.out.println(n); // boucle foreach
 }
}
...

```

#### Détails :

- Les blocs sont délimités par { }.
- La boucle for classique : init; condition; incrément.
- La boucle for-each (for (int n : nombres)) itère sur un tableau ou une collection.
- compteur++ : incrémentation postfixée.

### 3.4 Programmation orientée objet détaillée

#### Encapsulation et constructeurs

```

```java

```

```

public class Personne {

```

```

private String nom; // attribut privé : encapsulation
private int age;
// Constructeur
public Personne(String nom, int age) {
    this.nom = nom;
    this.age = age;
}
// Getter
public String getNom() {
    return nom;
}
// Setter
public void setNom(String nom) {
    this.nom = nom;
}
public void sePresenter() {
    System.out.println("Je m'appelle " + nom + ", j'ai " + age + " ans.");
}
}
...

```

- private : l'attribut n'est accessible que depuis la classe elle-même.
- this.nom : référence l'attribut de l'instance (permet de distinguer du paramètre nom).
- Les getters/setters permettent un accès contrôlé.

Héritage et polymorphisme

```

```java
// Classe de base
public class Animal {
 protected String nom; // protected : accessible aux sous-classes

```

```

public Animal(String nom) {
 this.nom = nom;
}

public void crier() {
 System.out.println(nom + " fait un bruit.");
}
}

// Classe dérivée
public class Chien extends Animal {
 public Chien(String nom) {
 super(nom); // appel au constructeur parent
 }

 @Override // annotation (optionnelle) indique qu'on redéfinit
 public void crier() {
 System.out.println(nom + " aboie !");
 }

 // Surcharge (overloading) : même nom, paramètres différents
 public void crier(int fois) {
 for (int i = 0; i < fois; i++) {
 System.out.println(nom + " aboie !");
 }
 }
}
...

```

- protected : accessible dans le package et les sous-classes.
- super(nom) : appelle le constructeur de la classe parente.
- L'annotation @Override n'est pas obligatoire mais recommandée pour détecter des erreurs.

## Collections

```
```java
import java.util.*;

public class CollectionsDemo {

    public static void main(String[] args) {

        // ArrayList : liste dynamique

        List<String> fruits = new ArrayList<>();

        fruits.add("Pomme");
        fruits.add("Banane");
        fruits.add("Orange");

        for (String fruit : fruits) {

            System.out.println(fruit);

        }

        // HashMap : dictionnaire

        Map<String, Integer> ages = new HashMap<>();

        ages.put("Alice", 25);
        ages.put("Bob", 30);

        int ageAlice = ages.get("Alice"); // 25

        System.out.println("Âge d'Alice : " + ageAlice);

    }

}
```

- List<String> : interface, ArrayList est une implémentation.
- Les génériques (<String>) garantissent la sécurité de type.
- Map<String, Integer> associe des clés de type String à des valeurs de type Integer.

3.5 Gestion des exceptions

```
```java

public class ExceptionDemo {
```

```

public static double diviser(double a, double b) throws ArithmeticException {
 if (b == 0) {
 throw new ArithmeticException("Division par zéro");
 }
 return a / b;
}

public static void main(String[] args) {
 try {
 double resultat = diviser(10, 0);
 System.out.println(resultat);
 } catch (ArithmeticException e) {
 System.out.println("Erreur : " + e.getMessage());
 } finally {
 System.out.println("Bloc finally toujours exécuté");
 }
}
}
...

```

- throws dans la signature indique que la méthode peut lancer une exception (non obligatoire pour les exceptions non vérifiées comme ArithmeticException).
- throw : lance explicitement une exception.
- try-catch-finally : finally s'exécute quoi qu'il arrive (même si un return est dans le try ou le catch).

### 3.6 JDBC (Java Database Connectivity) – étape par étape

**Prérequis** : ajouter le driver JDBC de H2 (penser à le mettre dans le classpath).

```

```java
import java.sql.*;

public class JdbcDemo {
    // URL de connexion à une base H2 embarquée (fichier ~/test)

```



```

private static final String URL = "jdbc:h2:~/test";
private static final String USER = "sa";
private static final String PASSWORD = "";
public static void main(String[] args) {

    // Utilisation du try-with-resources pour fermer automatiquement Connection, Statement,
    ResultSet

    try (Connection conn = DriverManager.getConnection(URL, USER, PASSWORD);
        Statement stmt = conn.createStatement()) {

        // Création de la table

        String sqlCreate = "CREATE TABLE IF NOT EXISTS etudiants (" +
            "id INT PRIMARY KEY, " +
            "nom VARCHAR(100), " +
            "age INT)";

        stmt.executeUpdate(sqlCreate); // executeUpdate pour INSERT, UPDATE, DELETE, DDL

        // Insertion

        String sqlInsert = "INSERT INTO etudiants VALUES (1, 'Alice', 25)";
        stmt.executeUpdate(sqlInsert);

        // Lecture

        String sqlSelect = "SELECT * FROM etudiants";
        ResultSet rs = stmt.executeQuery(sqlSelect); // executeQuery pour SELECT

        while (rs.next()) {

            int id = rs.getInt("id");

            String nom = rs.getString("nom");

            int age = rs.getInt("age");

            System.out.println(id + ". " + nom + " - " + age + " ans");

        }

    } catch (SQLException e) {

        e.printStackTrace();

    }
}

```

```
}  
}  
...
```

Explications :

- DriverManager.getConnection() établit la connexion avec la base.
- Statement permet d'exécuter des requêtes SQL. Pour éviter les injections SQL, on préférera PreparedStatement (voir exercice).
- ResultSet représente un curseur sur les résultats. rs.next() avance à la ligne suivante et retourne false s'il n'y a plus de lignes.
- try-with-ressources : les ressources déclarées entre parenthèses après try sont automatiquement fermées (appel de close()) à la fin du bloc.

3.7 Exercices pratiques avec corrigés détaillés

Exercice 1 : Classe Etudiant

Corrigé commenté :

```
```java  
public class Etudiant {
 private String nom;
 private String prenom;
 private double[] notes;
 public Etudiant(String nom, String prenom) {
 this.nom = nom;
 this.prenom = prenom;
 this.notes = new double[0]; // tableau vide par défaut
 }
 public void setNotes(double[] notes) {
 this.notes = notes;
 }
 public double moyenne() {
 if (notes.length == 0) return 0;
 }
}
```

```

 double somme = 0;
 for (double note : notes) {
 somme += note;
 }
 return somme / notes.length;
}

public void afficherInfos() {
 System.out.println(prenom + " " + nom + " - Moyenne: " + moyenne());
}

public static void main(String[] args) {
 Etudiant e = new Etudiant("Dupont", "Alice");
 double[] tabNotes = {14.5, 16.0, 12.5};
 e.setNotes(tabNotes);
 e.afficherInfos(); // Alice Dupont - Moyenne: 14.333...
}
}
...

```

### Explications supplémentaires :

- notes.length donne la taille du tableau.
- La boucle for (double note : notes) itère sur chaque note.

### Exercice 2 : CRUD avec PreparedStatement

Énoncé : Créer une classe ProduitDAO qui permet d'ajouter et lister des produits.

Extrait important (utilisation de PreparedStatement pour éviter les injections SQL) :

```

```java
public void ajouter(String nom, double prix) throws SQLException {
    String sql = "INSERT INTO produits (nom, prix) VALUES (?, ?)";
    try (PreparedStatement pstmt = connection.prepareStatement(sql)) {
        pstmt.setString(1, nom); // le premier '?' reçoit la valeur de nom
    }
}

```

```
        pstmt.setDouble(2, prix);  
        pstmt.executeUpdate();  
    }  
}  
...
```

- PreparedStatement précompile la requête et échappe les caractères dangereux.

Chapitre 4 : Programmation JEE (Servlets et JSP)

Objectifs détaillés

- Comprendre le rôle d'un serveur web (Tomcat) et l'architecture MVC.
- Écrire une Servlet qui répond à des requêtes HTTP GET/POST.
- Créer des pages JSP avec des éléments dynamiques.
- Utiliser les attributs de requête, de session et l'expression language (EL).
- Déployer une application web simple (structure de dossiers, web.xml ou annotations).

4.1 Architecture JEE et modèle MVC

MVC (Modèle-Vue-Contrôleur) est un patron d'architecture :

- Modèle : les données (ex : une classe Etudiant, un DAO).
- Vue : la présentation (fichiers JSP, HTML).
- Contrôleur : la servlet qui reçoit la requête, interagit avec le modèle, puis choisit la vue à afficher.

Un serveur comme Apache Tomcat interprète les servlets et les JSP.

4.2 Votre première Servlet – explication ligne par ligne

```
```java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {
 @Override
 protected void doGet(HttpServletRequest request,
 HttpServletResponse response)
 throws ServletException, IOException {
 response.setContentType("text/html");
 PrintWriter out = response.getWriter();
 }
}
```

```

out.println("<!DOCTYPE html>");
out.println("<html>");
out.println("<head><title>Ma Servlet</title></head>");
out.println("<body>");
out.println("<h1>Bonjour depuis ma servlet !</h1>");
out.println("</body>");
out.println("</html>");

}

}

...

```

### Détails :

- extends HttpServlet : notre classe hérite de la classe de base pour les servlets HTTP.
- doGet : méthode appelée quand la requête HTTP est de type GET (par exemple, clic sur un lien, saisie d'URL). Il existe aussi doPost, doPut, etc.
- HttpServletRequest request : contient les paramètres, les en-têtes, les cookies, la session.
- HttpServletResponse response : permet de construire la réponse (statut HTTP, en-têtes, corps).
- response.setContentType("text/html") : indique au navigateur que la réponse est du HTML.
- PrintWriter out = response.getWriter() : obtient un flux pour écrire le corps de la réponse.
- On écrit le code HTML directement dans la servlet (c'est mal pratique, d'où l'intérêt des JSP ensuite).

Déploiement : Pour que Tomcat trouve cette servlet, on peut la déclarer avec une annotation @WebServlet("/hello") en début de classe :

```

``java
@WebServlet("/hello")
public class HelloServlet extends HttpServlet { ... }

...

```

Ainsi, toute requête `http://localhost:8080/monapp/hello` sera dirigée vers cette servlet.

### 4.3 Traitement d'un formulaire – explications

Formulaire HTML (inscription.jsp) :

```
```jsp
<form action="inscription" method="post">
    <label>Nom : <input type="text" name="nom"></label>
    <label>Email : <input type="email" name="email"></label>
    <input type="submit">
</form>
```
```

Servlet InscriptionServlet (annotée @WebServlet("/inscription")) :

```
```java
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String nom = request.getParameter("nom");
    String email = request.getParameter("email");
    request.setAttribute("message",
        "Merci " + nom + " ! Votre email " + email + " a été enregistré.");
    RequestDispatcher dispatcher =
        request.getRequestDispatcher("/confirmation.jsp");
    dispatcher.forward(request, response);
}
```
```

#### Explications :

- request.getParameter("nom") récupère la valeur du champ nom envoyé dans le formulaire.
- request.setAttribute("message", ...) : attache un objet (ici une chaîne) à la requête sous la clé "message". Cet attribut pourra être lu par la JSP de destination.
- RequestDispatcher : objet qui permet de transférer la requête (et la réponse) à une autre ressource (JSP, servlet) sur le même serveur.
- forward() : le contrôle est passé à confirmation.jsp. Le client (navigateur) n'est pas redirigé, l'URL reste celle de la servlet.

## 4.4 JSP avec Expression Language (EL)

**confirmation.jsp :**

```
```.jsp
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head><title>Confirmation</title></head>
<body>
    <h1>Confirmation</h1>
    <p>${message}</p>    <!-- Affiche l'attribut "message" de la requête -->
    <a href="inscription.jsp">Retour</a>
</body>
</html>
...

```

· `${message}` est l'Expression Language (EL). Elle cherche d'abord dans les attributs de requête, puis de session, puis de contexte. Cela évite les scripts Java (`<% ... %>`) qui sont déconseillés.

4.5 JSTL – boucles et conditions

JSTL (JavaServer Pages Standard Tag Library) fournit des balises pour les conditions, boucles, formatage.

Ajouter le taglib dans la JSP :

```
```.jsp
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
...

```

Ensuite, supposons qu'une servlet ait placé une liste d'étudiants dans la requête :

```
```.java
List<Etudiant> liste = dao.lister();
request.setAttribute("etudiants", liste);
...

```


Dans la JSP, on peut itérer :

```
``jsp
<table border="1">
  <tr><th>ID</th><th>Nom</th><th>Âge</th></tr>
  <c:forEach var="etudiant" items="${etudiants}">
    <tr>
      <td>${etudiant.id}</td>
      <td>${etudiant.nom}</td>
      <td>${etudiant.age}</td>
    </tr>
  </c:forEach>
</table>
``
```

· c:forEach : boucle. L'attribut var est le nom de la variable qui prend successivement chaque élément de la collection pointée par items.

· Pour que \${etudiant.id} fonctionne, la classe Etudiant doit avoir une méthode getId() (un getter public).

4.6 Sessions et cookies – explications

Session : permet de conserver des données d'un utilisateur entre plusieurs requêtes.

```
``java
HttpSession session = request.getSession();
session.setAttribute("utilisateur", nom);
session.setMaxInactiveInterval(30 * 60); // 30 minutes
``
```

Cookie : stockage côté client.

```
``java
Cookie cookie = new Cookie("nom_utilisateur", nom);
cookie.setMaxAge(60 * 60 * 24 * 7); // 7 jours
```

```
response.addCookie(cookie);
```

```
...
```

4.7 Exercice pratique – Mini application CRUD des étudiants

Structure de l'application :

- Etudiant.java : modèle (id, nom, age).
- EtudiantDAO.java : accès base de données.
- ListeEtudiantsServlet.java : récupère la liste et la passe à listeEtudiants.jsp.
- ajouterEtudiant.jsp : formulaire d'ajout.
- AjouterEtudiantServlet.java : traite le POST, ajoute en base, redirige vers la liste.

Extrait de ListeEtudiantsServlet :

```
```java
```

```
@WebServlet("/liste")
```

```
public class ListeEtudiantsServlet extends HttpServlet {
```

```
 protected void doGet(HttpServletRequest request, HttpServletResponse response)
```

```
 throws ServletException, IOException {
```

```
 try {
```

```
 EtudiantDAO dao = new EtudiantDAO();
```

```
 List<Etudiant> etudiants = dao.lister();
```

```
 request.setAttribute("etudiants", etudiants);
```

```
 request.getRequestDispatcher("/WEB-INF/listeEtudiants.jsp").forward(request, response);
```

```
 } catch (SQLException e) {
```

```
 throw new ServletException(e);
```

```
 }
```

```
}
```

```
}
```

```
...
```

Remarque : Placer les JSP dans /WEB-INF/ empêche un accès direct par URL : elles ne peuvent être appelées que via une servlet. C'est une bonne pratique.

Projet final intégrateur (à rendre après la dernière séance)

## Dashboard système et utilisateurs

Objectif : développer une application complète en quatre étapes, chacune correspondant à un langage vu.

1. Script shell : collecte les métriques système (CPU, RAM, disque) toutes les 5 minutes (via cron) et les écrit dans un fichier metrics.json. Le script doit utiliser top, free, df et formater la sortie en JSON.
2. Script Python : lit metrics.json, effectue éventuellement des calculs (moyenne, max) et génère un fichier metrics.csv avec colonnes : timestamp, cpu\_usage, ram\_usage, disk\_usage.
3. Application Java : lit metrics.csv et insère les lignes dans une base de données SQLite (ou H2) table metrics.
4. Application web JEE : une servlet lit la table metrics et affiche dans une JSP un tableau HTML des mesures, avec possibilité de filtrer par date.

Livrables : code source complet, script de déploiement (ou instructions), rapport d'une page expliquant l'enchaînement des traitements et les difficultés rencontrées.

## **Bibliographie et ressources en ligne**

- Shell : Bash Guide for Beginners (gratuit)
- Python : Python officiel – tutoriel ; OpenClassrooms “Apprenez à programmer en Python”
- Java : Head First Java (Kathy Sierra, Bert Bates) ; Cours Java de l’Université de Lille
- JEE : Servlets et JSP – OpenClassrooms ; Oracle Java EE Tutorial